

OpenCV Camera Vision for Robotics

Detailed Report of the Face Detection, Eye Detection, Camera Calibration, Plane Mapping, Measurement, and Robotics Vision Discussion

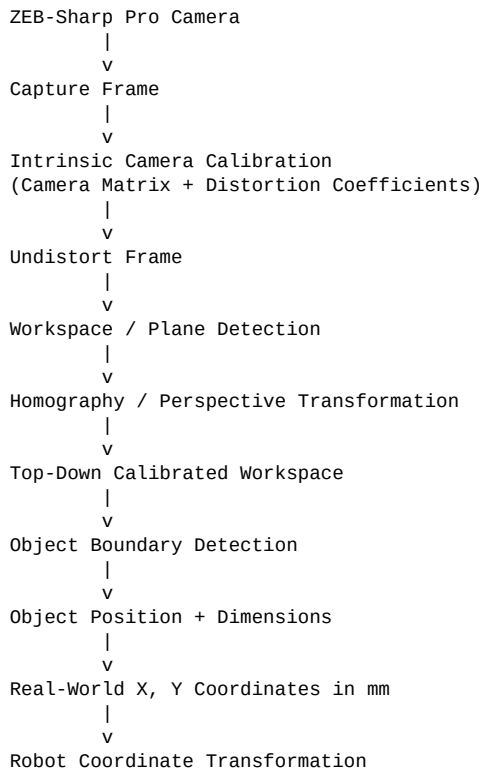
Project context: A Python virtual environment using OpenCV with a Zebronics ZEB-Sharp Pro camera, progressing from webcam face detection toward a calibrated robotics vision system.

Report scope: This document consolidates the technical discussion and recommendations developed in the conversation so far. It is intended as a practical reference for continuing the robotics project.

1. Executive Summary

The project began with a basic OpenCV webcam application and successfully progressed through face detection and eye detection. The discussion then expanded into camera calibration for robotics, including lens distortion correction, checkerboard calibration, perspective correction, real-world measurement, plane mapping, object boundary detection, and future conversion into robot coordinates.

The recommended final architecture is:



2. Development Environment

The project is being developed in Python inside a virtual environment. OpenCV is the primary computer-vision library.

```
pip install opencv-python
```

The OpenCV installation can be verified with:

```
python -c "import cv2; print(cv2.__version__)"
```

3. Initial Face Detection System

The first working program opened the default webcam using `cv2.VideoCapture(0)`, converted frames to grayscale, and used OpenCV's pre-trained Haar Cascade classifier to detect faces.

```
import cv2

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

cap = cv2.VideoCapture(0)

while True:
    success, frame = cap.read()
    if not success:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

for (x, y, w, h) in faces:
    cv2.rectangle(
        frame,
        (x, y),
        (x + w, y + h),
        (255, 0, 0),
        2
    )

cv2.imshow("Face Detection", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

Important concepts:

- VideoCapture(0) accesses the default webcam.
- Grayscale conversion reduces the information needed by the Haar classifier.
- detectMultiScale() searches the image at multiple scales.
- cv2.rectangle() highlights detected regions.
- OpenCV uses BGR color ordering rather than RGB.

4. Eye Detection and Highlighting

The next requirement was to detect eyes and draw red rectangular boundaries around them. Eye detection was restricted to the detected face region, which reduces false detections and unnecessary image searching.

```

eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_eye_tree_eyeglasses.xml"
)

face_gray = gray[y:y + int(h * 0.6), x:x + w]
face_color = frame[y:y + int(h * 0.6), x:x + w]

eyes = eye_cascade.detectMultiScale(
    face_gray,
    scaleFactor=1.05,
    minNeighbors=3,
    minSize=(20, 20)
)

for (ex, ey, ew, eh) in eyes:
    cv2.rectangle(
        face_color,
        (ex, ey),
        (ex + ew, ey + eh),
        (0, 0, 255),
        2
    )

```

The red color is (0, 0, 255) because OpenCV stores color values in BGR order. The eyeglasses cascade was suggested because it can be more tolerant of glasses than the basic eye cascade.

5. Why Camera Calibration Is Needed for Robotics

Basic detection only tells the program where something appears in the image. Robotics often requires physical information such as distance, position, dimensions, and orientation. Camera calibration provides the geometric information needed to connect image pixels with physical geometry.

The three major goals discussed were:

Goal	Required Technique
Correct lens distortion	Intrinsic camera calibration
Measure real-world size or position on a plane	Calibration + homography / scale mapping
Know plane or object pose relative to camera	Intrinsic calibration + pose estimation when applicable
Create a top-down view of a flat surface	Perspective transformation / homography
Convert workspace coordinates to robot coordinates	Camera-to-robot coordinate transformation

6. Intrinsic Camera Calibration

Intrinsic calibration estimates the internal parameters of the camera. The main results are a camera matrix and lens distortion coefficients.

Camera Matrix:

```
[ fx  0  cx ]
[  0 fy  cy ]
[  0  0   1 ]
```

- f_x and f_y are focal lengths expressed in pixel units.
- c_x and c_y represent the principal point or optical center estimate.
- The distortion coefficients commonly model radial and tangential distortion.

$[k_1, k_2, p_1, p_2, k_3]$

The recommended process is to capture multiple images of a known checkerboard from different positions, distances, and orientations. OpenCV detects the internal checkerboard corners and uses their known geometric layout to estimate the camera parameters.

7. Checkerboard Pattern Selection

The discussion considered a checkerboard from the Mark Hedley Jones calibration checkerboard collection. A practical recommendation was an A4 pattern with 25 mm squares and 10×7 internal corners, corresponding to 11×8 physical squares.

Property	Recommended Starting Value
Paper size	A4
Square dimensions	25 mm $\times$ 25 mm
Internal corners	10×7
Physical squares	11×8
OpenCV checkerboard tuple	(10, 7)

A critical distinction is that OpenCV's `findChessboardCorners()` uses the number of internal corners, not the number of black/white squares.

```
CHECKERBOARD = (10, 7)
SQUARE_SIZE = 25.0 # millimeters
```

Printing should be done at actual size / 100% scaling. The physical square size should then be measured. If the printed square is, for example, 24.8 mm rather than exactly 25 mm, using the measured value is preferable when

absolute physical measurements are required.

8. Capturing Calibration Images

A capture program was proposed to save checkerboard images when the space bar is pressed. Approximately 15–30 good images were recommended.

The checkerboard should not remain in nearly the same position. Good calibration images should include:

- Centered positions.
- Positions near the left, right, top, and bottom of the frame.
- Different distances from the camera.
- Tilted orientations.
- Different rotations.
- A range of useful viewing angles while keeping corners clearly visible.

This improves coverage of the camera sensor and helps estimate distortion across the image.

9. Calibration Code Concept

The calibration process creates known 3D object points on the checkerboard plane and matches them with detected 2D image corner locations.

```
objp = np.zeros(
    (CHECKERBOARD[0] * CHECKERBOARD[1], 3),
    np.float32
)

objp[:, :2] = np.mgrid[
    0:CHECKERBOARD[0],
    0:CHECKERBOARD[1]
].T.reshape(-1, 2)

objp *= SQUARE_SIZE
```

Detected corners are refined with `cornerSubPix()`, and calibration is performed with `cv2.calibrateCamera()`.

```
ret, camera_matrix, distortion_coefficients, rvecs, tvecs = cv2.calibrateCamera(
    object_points,
    image_points,
    image_size,
    None,
    None
)
```

The calibration results can be stored for later use:

```
np.savez(
    "camera_calibration.npz",
    camera_matrix=camera_matrix,
    distortion_coefficients=distortion_coefficients
)
```

10. Lens Distortion Correction

Once calibration is complete, each incoming frame can be corrected before further computer-vision processing.

```
data = np.load("camera_calibration.npz")

camera_matrix = data["camera_matrix"]
distortion_coefficients = data["distortion_coefficients"]

undistorted = cv2.undistort(
    frame,
```

```

        camera_matrix,
        distortion_coefficients
    )

```

For the robotics pipeline, subsequent operations such as line detection, contour detection, plane mapping, and object measurement should preferably use the undistorted frame.

11. Resolution and Camera Settings

Calibration parameters are associated with the camera configuration used during calibration. For best consistency, the project should use the same image resolution and comparable camera settings during calibration and normal operation.

```

cap = cv2.VideoCapture(0)

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

```

The actual resolution received by OpenCV can be checked with:

```

width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print(f"Actual camera resolution: {int(width)} x {int(height)}")

```

The Zebronics ZEB-Sharp Pro camera was identified as the camera being used. The practical recommendation is to select the resolution intended for the robotics application and keep that configuration consistent during calibration and deployment.

12. Object and Plane Boundary Detection

For simple image-space boundary detection, additional camera calibration is not strictly necessary. However, using the already calibrated and undistorted image improves geometric consistency, especially near the edges of the image.

The proposed processing chain is:

```

Undistorted Frame
    |
    v
Grayscale Conversion
    |
    v
Noise Reduction
    |
    v
Canny Edge Detection
    |
    v
Contour Extraction
    |
    v
Polygon Approximation
    |
    v
Boundary / Rectangle Detection

gray = cv2.cvtColor(
    undistorted,
    cv2.COLOR_BGR2GRAY
)

blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

edges = cv2.Canny(
    blurred,
    50,
    150
)

```

```

)

contours, _ = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

```

Polygon approximation can identify contours that approximately contain four vertices, which may represent rectangular surfaces or objects:

```

perimeter = cv2.arcLength(contour, True)

approx = cv2.approxPolyDP(
    contour,
    0.02 * perimeter,
    True
)

if len(approx) == 4:
    cv2.drawContours(
        undistorted,
        [approx],
        -1,
        (0, 0, 255),
        4
    )

```

13. Marked Line Detection

For explicit lines rather than closed boundaries, a line detector such as the probabilistic Hough transform can be used after edge detection. This is useful for detecting marked lanes, borders, table edges, or structured workspace lines.

```

lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi / 180,
    threshold=50,
    minLineLength=50,
    maxLineGap=10
)

```

The parameters must be tuned for the lighting, line thickness, camera resolution, and scene.

14. Perspective Transformation and Homography

A flat rectangular surface generally does not look rectangular when viewed from an angled camera. A perspective transformation can map the four observed corner points to a rectangular top-down representation.

```

source_points = np.float32([
    [x1, y1], # top-left
    [x2, y2], # top-right
    [x3, y3], # bottom-right
    [x4, y4]  # bottom-left
])

destination_points = np.float32([
    [0, 0],
    [1200, 0],
    [1200, 800],
    [0, 800]
])

matrix = cv2.getPerspectiveTransform(
    source_points,
    destination_points
)

top_view = cv2.warpPerspective(
    undistorted,
    matrix,
    (1200, 800)
)

```

The resulting image is a normalized view of the flat workspace. This is the foundation for converting object positions into a consistent plane coordinate system.

15. Pixels to Millimeters

Suppose the physical workspace is known to be 600 mm wide and 400 mm high, and the normalized top-down output is 1200 × 800 pixels.

```
Workspace:  
600 mm × 400 mm
```

```
Top-down image:  
1200 px × 800 px
```

The scale is then:

```
mm_per_pixel_x = 600 / 1200 # 0.5 mm/px  
mm_per_pixel_y = 400 / 800  # 0.5 mm/px
```

An object detected at a pixel location can therefore be converted to plane coordinates:

```
real_x_mm = pixel_x * mm_per_pixel_x  
real_y_mm = pixel_y * mm_per_pixel_y
```

Similarly, object dimensions measured in the top-down image can be converted into millimeters.

16. Object Boundary, Center, and Size Measurement

After the workspace has been transformed into a top-down view, contour detection can be applied to objects placed on the plane.

```
for contour in contours:  
  
    area = cv2.contourArea(contour)  
  
    if area < 1000:  
        continue  
  
    x, y, w, h = cv2.boundingRect(contour)  
  
    center_x = x + w / 2  
    center_y = y + h / 2  
  
    cv2.rectangle(  
        top_view,  
        (x, y),  
        (x + w, y + h),  
        (0, 0, 255),  
        2  
    )
```

With a known scale, dimensions can be calculated as:

```
object_width_mm = w * mm_per_pixel_x  
object_height_mm = h * mm_per_pixel_y  
  
object_x_mm = center_x * mm_per_pixel_x  
object_y_mm = center_y * mm_per_pixel_y
```

For rotated objects, a minimum-area rotated rectangle or a more detailed contour measurement can be preferable to a simple axis-aligned bounding rectangle.

17. Workspace Detection: Automatic Versus Manual

Manually entering four corner coordinates is useful for initial testing but is fragile if the camera moves. Automatic workspace detection can use physical borders, strong rectangular edges, or visual markers.

A robust approach proposed for robotics is to place four uniquely identifiable ArUco markers at known workspace corners.

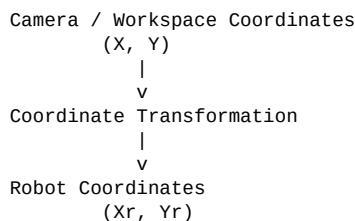


The intended automated process is:

- Detect the four markers.
- Identify which marker corresponds to each workspace corner.
- Extract their image coordinates.
- Calculate the homography.
- Transform the image to the calibrated top-down workspace.
- Detect objects within that normalized workspace.
- Convert object coordinates to millimeters.

18. Relationship to Robot Coordinates

A workspace coordinate system is not automatically identical to the robot's own coordinate system. A final transformation may be required to convert workspace coordinates into coordinates meaningful to the robot controller.



This transformation depends on how the camera and robot are physically mounted. If the camera or robot relationship changes, this calibration or transformation may need to be repeated.

19. Recommended Calibration Levels

Level	Purpose	Repeat When
1. Intrinsic camera calibration	Estimate camera matrix and lens distortion	Camera, resolution, or relevant optical configuration changes
2. Workspace / plane calibration	Map image to a known flat plane and physical units	Camera or workspace geometry changes
3. Camera-to-robot calibration	Map workspace coordinates to robot coordinates	Camera/robot physical relationship changes

20. Recommended Software Architecture

```
robot_vision/
|
|-- calibration/
|   |-- capture_checkerboard.py
|   |-- calibrate_camera.py
|   |-- camera_calibration.npz
|
|-- workspace/
|   |-- detect_workspace.py
|   |-- homography.npz
|
|-- detection/
|   |-- detect_objects.py
|
|-- main.py
```

The final processing logic can be represented as:

```
frame = get_camera_frame()

undistorted = undistort_camera(frame)

top_view = transform_workspace(undistorted)

objects = detect_objects(top_view)

for obj in objects:
    x_mm, y_mm = convert_to_mm(obj.center)
    width_mm, height_mm = get_dimensions(obj)

    print(
        x_mm,
        y_mm,
        width_mm,
        height_mm
    )
```

21. Recommended Implementation Roadmap

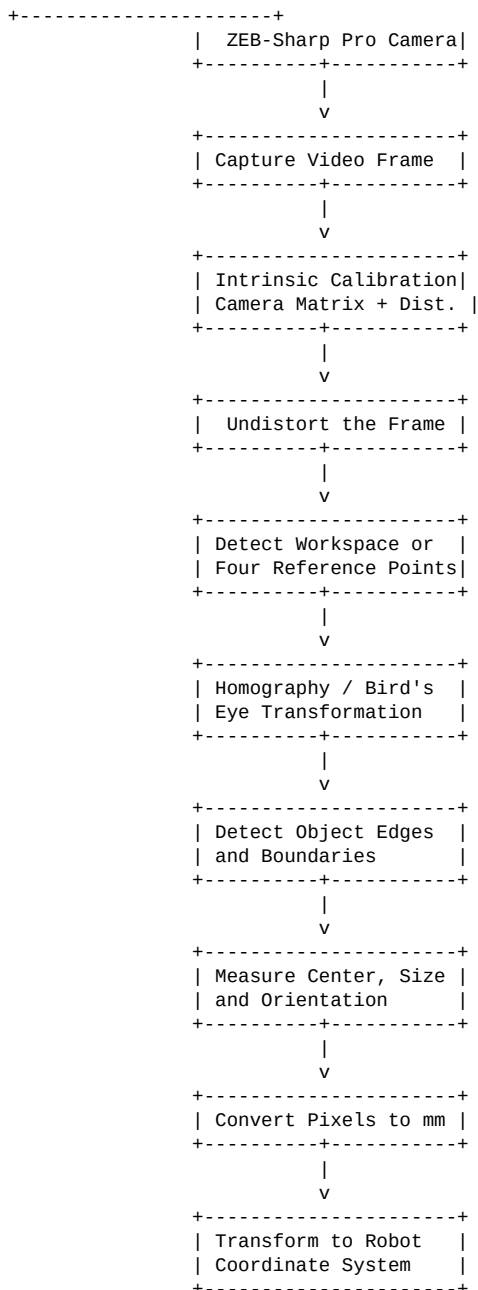
Phase	Task
1	Confirm stable camera capture and select the operating resolution.
2	Print the checkerboard at 100% scale and measure actual square dimensions.
3	Capture 20–30 high-quality calibration images.
4	Run intrinsic calibration and save camera_matrix and distortion_coefficients.
5	Calculate and inspect calibration quality, including reprojection error and visual undistortion tests.
6	Create a live undistorted camera program.
7	Create a known rectangular workspace.
8	Detect or mark the four workspace corners.
9	Calculate and save the homography.
10	Generate a stable top-down workspace image.
11	Define the physical workspace dimensions in millimeters.
12	Detect object boundaries and calculate object center and dimensions.
13	Convert detected measurements into workspace X/Y coordinates.
14	Calibrate the relationship between workspace coordinates and robot coordinates.
15	Integrate the resulting coordinates with robot control logic.

22. Key Practical Notes

- Face and eye Haar cascades are useful learning tools but are older detection methods and may be sensitive to lighting and pose.
- For accurate robotics work, perform intrinsic calibration before relying on geometric measurements.
- Use the same camera resolution during calibration and deployment whenever possible.
- Print calibration patterns without automatic scaling and measure the actual physical square size.
- Collect calibration images with broad positional and angular variation.
- Undistort frames before performing geometric workspace mapping.
- Homography-based measurement assumes the measured objects lie on the calibrated plane.

- Objects above the plane can introduce perspective and height-related measurement errors.
- Automatic workspace markers can be more reliable than trying to infer table edges under changing lighting conditions.
- A separate coordinate transformation is generally required to map camera/workspace coordinates into robot coordinates.

23. Final System Concept



24. Conclusion

The project has a clear progression from basic computer vision to a robotics-ready measurement system. The immediate foundation is intrinsic camera calibration and distortion correction. The next major milestone is defining a known workspace and calculating a homography so that every point on that flat surface can be expressed in a consistent coordinate system. Once that is working, object boundaries can be detected and converted into real-world dimensions and positions. The final robotics stage is establishing the transformation from workspace coordinates to the robot's coordinate frame.

This approach is modular: each stage can be tested independently before integration into the final robotic system.